

PrepME AI — Day 3 Backend Architecture Notes

1. Revision of Current Full Stack Flow

Current application flow:

```
Frontend (React)
  ↓
Axios Request
  ↓
Backend Route
  ↓
Controller Logic
  ↓
MongoDB Model
  ↓
Database Storage
  ↓
Response Sent Back
  ↓
Frontend UI Update
```

This is the core architecture of modern full-stack applications.

2. What We Achieved Today

Today we:

- created User schema
 - created User model
 - saved real users into MongoDB
 - learned documents and collections
 - separated backend into multiple files/folders
 - introduced backend architecture
 - learned routes/controllers separation
 - built more professional project structure
-

3. MongoDB Concepts Revision

Database

Purpose:

- permanent data storage

Examples:

- users
- passwords
- notes
- progress

Without database:

- data disappears after server restart
-

Collection

Collection = group of documents.

Example:

```
users
```

Similar to table in SQL databases.

Document

Single stored item.

Example:

```
{  
  "name": "Shweta",  
  "email": "abc@gmail.com"  
}
```

MongoDB stores data in document format.

Schema

Schema defines structure of document.

Example:

```
const userSchema = new mongoose.Schema({  
  name: String,  
  email: String,  
  password: String  
})
```

Meaning: Every user should contain:

- name
 - email
 - password
-

Model

Model is created from schema.

Example:

```
const User = mongoose.model("User", userSchema)
```

Purpose:

- interact with database
 - save data
 - fetch data
 - delete data
 - update data
-

save()

Example:

```
await newUser.save()
```

Purpose:

- store document in MongoDB

This is the actual database write operation.

4. Understanding Stored MongoDB Data

Example document:

```
_id
6a173d254a2d91299c3e9002

name
"Shweta"

email
"shwetasingh@gmail.com"

password
"abc@123"

__v
0
```

_id

Automatically generated unique identifier.

Purpose:

- uniquely identify document
- primary identity of data

MongoDB automatically creates this.

__v

Automatically added by Mongoose.

Version key used internally.

Can mostly be ignored for now.

5. Why We Needed Better Folder Structure

Initially everything was inside:

```
index.js
```

Problems with this approach:

- messy code
- difficult debugging
- hard to scale
- difficult maintenance

Works for small demos. Not good for real applications.

6. Real Backend Architecture

Modern backend applications separate responsibilities.

Different files handle different purposes.

This makes:

- code cleaner
 - debugging easier
 - scaling easier
 - collaboration easier
-

7. Final Backend Folder Structure

```
server
|
├── models
|   └── User.js
|
├── routes
|   └── userRoutes.js
|
├── controllers
|   └── userController.js
|
├── config
|   └── db.js
|
├── .env
└── index.js
```

This is a beginner-friendly professional backend structure.

8. models Folder

Purpose:

- define database structure
- create schemas
- create models

Examples:

- User model
- Note model
- Progress model

Example file:

```
models/User.js
```

What Happens Inside models?

Example:

```
const userSchema = new mongoose.Schema({
  name: String,
  email: String,
  password: String
})
```

This defines:

How user data should look in database

9. routes Folder

Purpose:

- define API endpoints
- connect requests to controllers

Examples:

- /login
- /signup
- /users

Example file:

routes/userRoutes.js

express.Router()

Example:

```
const router = express.Router()
```

Purpose:

- create modular routes
- separate APIs into files

Very important backend scaling concept.

Route Example

```
router.post("/login", loginUser)
```

Meaning:

```
If POST request comes to /login,  
run loginUser controller.
```

10. controllers Folder

Purpose:

- business logic layer
- actual backend operations

Example file:

```
controllers/userController.js
```

What Controllers Do

Examples:

- save users
- validate login
- fetch notes
- update progress
- delete data

Controllers contain application logic.

Example Controller

```
const loginUser = async (req, res) => {  
  
  const newUser = new User({  
    name: "Shweta",  
    email: req.body.email,  
    password: req.body.password  
  })  
  
  await newUser.save()  
  
  res.json({  
    message: "User saved"  
  })  
}
```

11. config Folder

Purpose:

- application configuration
- reusable setup logic

Example:

```
config/db.js
```

What db.js Does

Contains database connection logic.

Example:

```
mongoose.connect(process.env.MONGO_URI)
```

Purpose:

- connect backend to MongoDB

Separating config makes code cleaner.

12. index.js Role

After restructuring:

```
index.js
```

became much cleaner.

Main responsibilities now:

- create Express app
 - apply middleware
 - connect DB
 - register routes
 - start server
-

13. app.use()

Example:

```
app.use("/api/users", userRoutes)
```

Meaning:

- all routes inside userRoutes start with:

```
/api/users
```

Route Combination Example

Inside routes:

```
router.post("/login")
```

Final endpoint becomes:

```
/api/users/login
```

Very important routing understanding.

14. Backend Request Flow

Current backend flow:

```
Frontend Request
  ↓
Express Route
  ↓
Controller Function
  ↓
Mongoose Model
  ↓
MongoDB Database
  ↓
Response Back To Frontend
```

This is real backend architecture.

15. Important Separation of Responsibilities

Routes

Responsible for:

Which controller should run?

Controllers

Responsible for:

What actual logic should happen?

Models

Responsible for:

How data should look?

Config

Responsible for:

Application setup/configuration

16. async/await Revision

Database operations take time.

Example:

```
await newUser.save()
```

Meaning:

Wait until database operation finishes.

17. try/catch Revision

Used for backend error handling.

Example:

```
try {  
  
}  
catch(error) {  
  
}
```

Purpose:

- prevent server crashes
- handle DB/API errors safely

Very important backend practice.

18. Current Application Status

Completed:

- React frontend
 - Express backend
 - API communication
 - MongoDB connection
 - User schema/model
 - Save users in DB
 - Backend architecture restructuring
 - Routes/controllers separation
-

19. Most Important Learning Today

Modern backend applications are NOT written in one file.

Professional applications separate:

- routes
- controllers
- models
- configuration

This improves:

- scalability
 - maintainability
 - readability
 - debugging
 - team collaboration
-

20. Next Learning Goals

Next steps:

- proper signup flow
- duplicate email checking
- password hashing using bcrypt
- validation
- authentication system
- JWT tokens

This is where backend starts becoming production-like.